15 Years of Sorting Competitions

Elena Machkasova

University of Minnesota, Morris

Midwest Instruction and Computing Symposium, March 27th 2026

15 years ago...

MICS 2011:

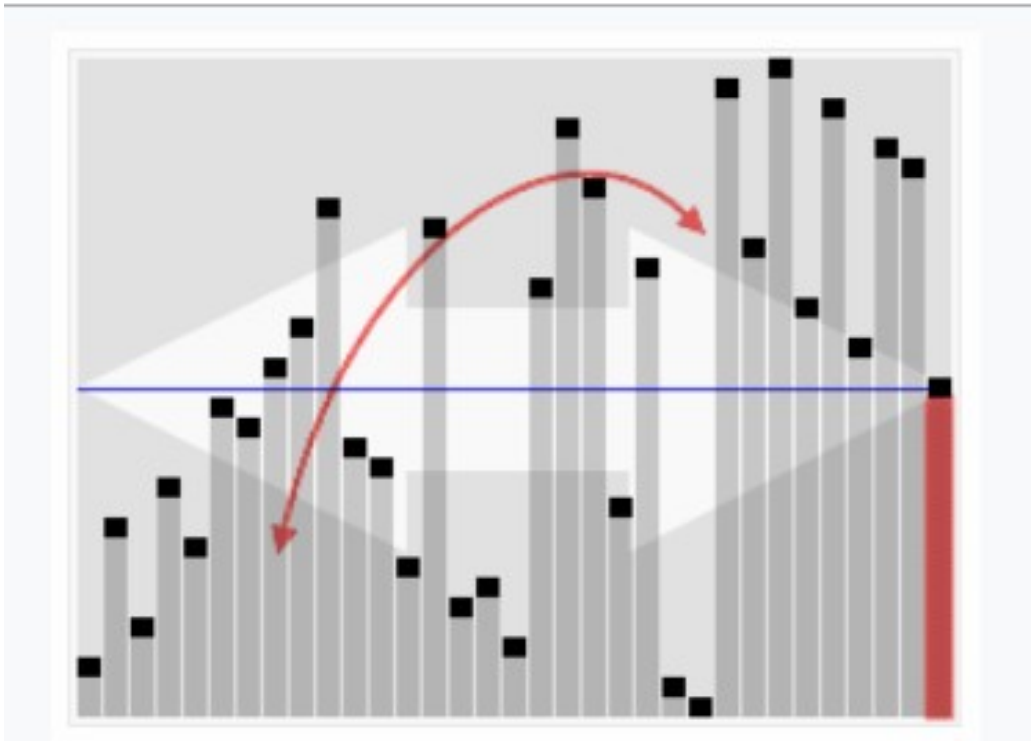
- Paper on a novel assignment in *Algorithms and Computability* class
- Typically students implement sorting on numbers and study O efficiency
- The assignment compared sorting by **time**
- Task: sort strings by length, and lexicographically within each length
- The assignment continued to be offered every year, with a different sorting problem every time

Outline

1. Overview and Setup
2. Sorting Problems
3. Correctness
4. Community Matters
5. Conclusions

Why sorting?

- No one perfect algorithm - lots of things to try
- Details matter! - problem solving



Source: https://en.wikipedia.org/wiki/Quicksort

Setup (summary)

- Programming in Java
- Timed portion: only sorting
- Template: file input/output
- Reference Java implementation: slow, correct
- Correctness: exact match to the output of reference implementation
- Data generator, usually with parameters (number of items, length of strings, etc.)
- Single core run
- JVM warmup provided

Scoring and logistics

- 2 preliminary rounds, 1 final round
- Compiled code (but not source code) is posted after each preliminary round
- Groups are anonymous, identified by numbers
- After the final round all source code is posted, groups revealed
- Each round has 2-3 datasets to sort, the score is the sum of places
- Each solution is ran 3 times on each dataset, the median time counts
- Incorrect sorting gets penalty of 1000000ms.

Some past problems (omitting details)

- 2014: binary strings to be sorted by length, then by the number of 1s
- 2016: 2D points on a grid generated as "walks' to be sorted by the distance to the closest of two constant points
- 2017: positive integers to be sorted by the product of two smallest different prime factors
- 2021: binary numbers to be sorted by the number of 1s, then by the length of the longest repeated substring
- 2022: mix of decimal fractions and rational numbers to be sorted by value. For equal numbers the decimal representation is considered "smaller" than a fraction for positives and "larger" for negatives
- 2023: Fixed length integers to be sorted by the sum of all unique prime factors. Equal sums are sorted by value in decreasing order
- 2025: Binary strings of the same length are to be sorted by their distance from a "target" string.

Important aspects of problems

Approaches that are commonly tried (as combinations)

- Many duplicates – modified quicksort, counting sort, split into buckets
- Fixed length – radix sort
- Uniform distribution – variations of bucket sort
- Ordered subsequences ("walks") – mergesort, TimSort, insertion sort
- Fits into a long – quicksort or any predefined sort on longs
- Repeated computations (e.g. finding prime factors) – storing results
- Multiple sorting paths – stable sorting matters, although "buckets" are also used
- Made appearance: heapsort, comb sort, prefix trees,....

Variety of solutions

- Every challenge had a variety of different solutions
- Often different approaches were among top three
- Some groups optimized the comparison itself, used Java sorting
- Quicksort works well, but typically needed to be customized for duplicates and other special cases; not stable
- Data representation matters: using primitive types (directly or as a part of a custom object) gives significant advantage
- Studying the data helps
- Optimizing for a common case is good (optimized vs fallback cases)
- Too ambitious solutions often resulted in correctness issues

Correctness review

A solution is correct if it follows all the rules (memory used, etc.) and produces correct sorting for *any* data that can be generated by the data generator.

- At the competition: compared to reference representation
- After the final round: every group is assigned a "review" group
- Extra credit: for anyone reporting a correctness issue in **any** submission before their "reviewer" group does
- A group not passing correctness review is disqualified
- Rules decisions need to be made for unusual solutions: multithreading

Open competition

- Everyone can participate
- Alumni, other students participate (had a couple of sophomores)
- Some alums program Java professionally
- All code is posted – students can learn from external solutions
- Some ideas for challenges came from former students
- It's fun!

Presentations

- Presentations are often attended by students out of class, faculty
- Students have an option of presenting on out-of-class solutions for extra credit (in addition to their own)
- All code and slides are posted on github
- It's a tradition

Conclusions and the future

- Sorting competition allows students explore algorithms, Java, program optimization
- It's a great problem solving experience
- It is a fun and exciting event
- It's a 15-year tradition that brings together students, alumni, and faculty
- Hope it continues for many years to come!
- Future issue: AI-proofing

Acknowledgments, more info

Thanks to Dr. Peter Dolan for including the sorting competitions into his offerings of the class and for collaborating on putting together the problems!
Thanks to students and alumni for keeping it interesting and fun!

Where can you find the competitions?
Before 2022:
`https://github.com/elenam/SortingCompetitionMaterials20XX`
2022 and after:
`https://github.com/elenam/SortingCompetitionMaterials-20XX`
(some are not there)

Questions are welcome!